

Использование языка программирования Julia для решения математических задач

Часть I

Авторы:

Чуракова Юлия Романовна

*Студентка факультета математики и компьютерных наук
Кубанского Государственного Университета*

Алексеев Евгений Ростиславович

*Кандидат технических наук, доцент
Доцент кафедры информационных технологий
Кубанского Государственного Университета*

Полная версия пособия доступна в репозитории:

<https://github.com/JuliaChurakova/Julia>

Литература

1. Энгхейм, Э. Julia в качестве второго языка / Э. Энгхейм. – Москва: ДМК Пресс, 2023. – 446 с.
2. The Julia Programming Language: сайт. – URL: <https://julialang.org> (дата обращения: 10.10.2025).
3. Белов, Г. В. Краткое описание языка программирования Julia и некоторые примеры его использования/ Г. В. Белов. – Москва: МГТУ им. Н.Э. Баумана, 2024. – 108 с.
4. Шеррингтон, М. Осваиваем язык Julia / М. Шеррингтон. – Москва: ДМК Пресс, 2017. – 416 с.
5. Антонюк, В. А. Язык Julia как инструмент исследователя / В. А. Антонюк. – Москва: Физический факультет МГУ им. М. В. Ломоносова, 2019. – 48 с.

Глава 9 Визуальное программирование

Визуальное программирование – это подход, при котором разработка программ происходит с использованием графических интерфейсов и визуальных элементов. Одним из популярных инструментов для создания таких интерфейсов является **GTK**, который поддерживает различные языки программирования, включая C, Python и Julia. В Julia для работы с GTK используется пакет **Gtk.jl**. Далее мы познакомимся с этим пакетом и его возможностями для создания оконных приложений, добавления кнопок, текстовых полей, панелей и других элементов управления, что позволяет легко создавать интерактивные интерфейсы прямо в Julia.

Для получения дополнительной информации о пакете см. [документацию](#).

9.1 Создание окна (GtkWindow) и кнопки (GtkButton)

Начнём с очень простого примера, в котором создадим пустое окно размером 400x200 пикселей и добавим к нему кнопку. Для этого нам понадобятся несколько ключевых функций из библиотеки **Gtk.jl**: **GtkWindow**, **GtkButton**, **push!** и **showall**. Давайте подробнее рассмотрим их использование и синтаксис:

1. **GtkWindow** – функция для создания окна с заданными размерами и заголовком;
2. **GtkButton** – функция для создания кнопки;

3. **push!** – функция для добавления виджетов в окно;
4. **showall** – функция для отображения окна и всех его компонентов.

In [5]: **using** Gtk

```
# Создаём окно с заголовком "Моя первая программа на Gtk.jl" и размерами 400x200
win = GtkWindow("Моя первая программа на Gtk.jl", 400, 200)

# Создаём кнопку с текстом "Нажми на меня"
b = GtkButton("Нажми на меня")

# Добавляем кнопку в окно
push!(win,b)

# Показываем все элементы окна
showall(win)
```

Out[5]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1,000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexexpand=FALSE, vexpand=FALSE, hexexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Моя первая программа на Gtk.jl", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=400, default-height=200, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-top-level-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)

Теперь мы расширим пример, чтобы кнопка действительно могла что-то делать. Чтобы сделать кнопку интерактивной (например, вызвать функцию при нажатии), можно добавить обработчик событий с помощью функции **signal_connect**. Синтаксис: **signal_connect(handler, widget, signal_name)**

Параметры:

1. **handler** – функция (или метод), которая будет вызвана при наступлении события. Эта функция должна принимать как минимум один аргумент – сам виджет, к которому привязан сигнал (**виджет** – это элемент интерфейса для взаимодействия с пользователем, например кнопка, текстовое поле);
2. **widget** – виджет, к которому привязывается обработчик событий;
3. **signal_name** – строка, указывающая на имя события, на которое мы хотим реагировать. Например, "clicked" для кнопки, "changed" для

текстового поля, "destroy" для окна.

Напишем программу, которая при нажатии на кнопку будет выводить сообщение в консоль.

```
In [6]: using Gtk

win = GtkWindow("Пример подключения обработчика событий", 400, 200)

b = GtkButton("Нажми на меня")
push!(win, b)

# Определяем функцию-обработчик, которая будет вызвана при нажатии на кнопку
function on_button_clicked(b)
    # Эта строка выводит сообщение в консоль, когда кнопка нажата
    println("Кнопка была нажата")
end

# Связываем сигнал "clicked" (событие нажатия на кнопку) с функцией-обработч
# signal_connect устанавливает, что при нажатии на кнопку b будет вызвана фу
signal_connect(on_button_clicked, b, "clicked")

showall(win)
```

```
Out[6]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible
=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE,
is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE,
receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=
FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opac
ity=1.000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL,
margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-
bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=FALSE, vexpand-
set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child,
type=GTK_WINDOW_TOPLEVEL, title="Пример подключения обработчика событий", role
=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-
width=400, default-height=200, destroy-with-parent=FALSE, hide-titlebar-when-
maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HI
NT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE,
accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=
GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-gr
ip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id
, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```

Кнопка была нажата

Функция **set_gtk_property!** в Gtk.jl используется для динамического изменения внешнего вида и поведения различных Gtk-виджетов во время работы приложения. Все виджеты в Gtk имеют набор свойств, которые можно изменять, например, текст, цвет, размер и поведение. Синтаксис функции **set_gtk_property!**: `set_gtk_property!(widget, property, value)`

Параметры:

1. **widget** – это Gtk-виджет, для которого вы хотите установить свойство;
 2. **property** – это имя свойства, которое вы хотите изменить (:label, :text, :visible и т. д.);
 3. **value** – значение, которое вы хотите установить для этого свойства.
- Тип значения зависит от типа свойства.

Пример 1. Изменение текста в кнопке.

Каждому виджету можно установить различные свойства. Например, чтобы изменить текст кнопки, используем свойство **:label**.

In [7]: **using** Gtk

```
win = GtkWindow("Изменение текста на кнопке", 400, 200)
b = GtkButton("Нажми на меня")

push!(win, b)

function on_button_clicked(b)
    # Изменяем текст на кнопке с помощью set_gtk_property!
    set_gtk_property!(b, :label, "Нажато")
end

signal_connect(on_button_clicked, b, "clicked")

showall(win)
```

Out[7]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1,000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Изменение текста на кнопке", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=400, default-height=200, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)

Пример 2. Изменение видимости виджета.

Некоторые свойства управляют видимостью или состоянием виджетов. Например, чтобы скрыть или показать виджет, можно использовать

СВОЙСТВО **:visible**.

```
In [8]: using Gtk

win = GtkWindow("Пример наглядности", 400, 200)

b = GtkButton("Нажми на меня")
push!(win, b)

function on_button_clicked(b)
    set_gtk_property!(b, :visible, false) # Скрываем кнопку
end

signal_connect(on_button_clicked, b, "clicked")

showall(win)
```

```
Out[8]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=
TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1.000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Пример наглядности", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=400, default-height=200, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```

Пример 3. Изменение размера окна.

Размер окна можно установить с помощью свойств **:width_request** и **:height_request**.

```
In [10]: using Gtk

win = GtkWindow("Пример размера")

b = GtkButton("Нажми на меня")
push!(win, b)

function on_button_clicked(b)
    set_gtk_property!(win, :width_request, 500) # Устанавливаем ширину окна
    set_gtk_property!(win, :height_request, 500) # Устанавливаем высоту окна
end
```

```
signal_connect(on_button_clicked, b, "clicked")

showall(win)
```

```
Out[10]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1.000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Пример размера", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=-1, default-height=-1, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```

Пример 4. Изменение состояния кнопки (активна/неактивна)

Можно управлять состоянием кнопки, например, сделать её неактивной с помощью свойства **:sensitive**.

```
In [11]: using Gtk

win = GtkWindow("Неактивная кнопка")

b = GtkButton("Нажми на меня")
push!(win, b)

function on_button_clicked(b)
    set_gtk_property!(b, :sensitive, false) # Делаем кнопку неактивной (невозможна)
end

signal_connect(on_button_clicked, b, "clicked")

showall(win)
```

```
Out[11]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1.000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexexpand=FALSE, vexpand=FALSE, hexexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Неактивная кнопка", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=-1, default-height=-1, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```

9.2 Макеты

Обычно в приложение требуется добавить более одного виджета. Для этого GTK предоставляет несколько виджетов для организации макета. Вместо использования точного позиционирования, виджеты макета в GTK используют подход, при котором виджеты выравниваются в контейнерах, таких как коробки и таблицы.

9.2.1 GtkBox

GtkBox – это один из самых простых и популярных виджетов для создания макета в GTK. Он позволяет организовать другие виджеты в порядке, либо по горизонтали, либо по вертикали. GtkBox упрощает расположение элементов в интерфейсе, избавляя от необходимости вручную управлять позиционированием.

Можно указать, как именно вы хотите выравнивать виджеты внутри GtkBox – по горизонтали или по вертикали:

- **GtkBox(:h)** – горизонтальное расположение;
- **GtkBox(:v)** – вертикальное расположение.

Вы можете управлять тем, как виджеты растягиваются внутри GtkBox с помощью параметров `expand`, `fill` и `padding`:

- **expand** – указывает, будет ли виджет расширяться, чтобы заполнить доступное пространство;

- **fill** – указывает, будет ли виджет растягиваться по размеру в том направлении, в котором он выровнен (горизонтально или вертикально);
- **padding** – добавляет отступы вокруг виджета.

Виджеты добавляются в GtkBox в определённом порядке, и порядок их добавления влияет на то, в какой последовательности они будут отображаться (слева направо или сверху вниз).

In [14]: **using** Gtk

```
# Создаем окно
win = GtkWindow("GtkBox пример", 400, 200)

# Создаем горизонтальную коробку (выравнивание по горизонтали)
box = GtkBox(:h) # :h означает горизонтальное выравнивание

# Создаем несколько кнопок
button1 = GtkButton("Кнопка 1")
button2 = GtkButton("Кнопка 2")
button3 = GtkButton("Кнопка 3")

# Добавляем кнопки в коробку
push!(box, button1, button2, button3)

# Добавляем коробку в окно
push!(win, box)

# Показываем окно
showall(win)
```

Out[14]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1,000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="GtkBox пример", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=400, default-height=200, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)

После того как виджеты были добавлены в GtkBox, мы можем обратиться к каждому из них по индексу и получить их свойства.

```
In [15]: length(box)# Получаем количество виджетов в GtkBox
```

```
Out[15]: 3
```

```
In [16]: get_gtk_property(box[1], :label, String) # Получаем текст первой кнопки
```

```
Out[16]: "Кнопка 1"
```

```
In [17]: get_gtk_property(box[2], :label, String) # Получаем текст второй кнопки
```

```
Out[17]: "Кнопка 2"
```

Предположим, что вы хотите, чтобы кнопка "Кнопка 3" заполнила доступное пространство, а между кнопками был отступ. Мы можем настроить свойства **expand** и **spacing**.

```
In [18]: using Gtk
```

```
win = GtkWindow("GtkBox пример", 400, 200)

# Создаем горизонтальную коробку (выравнивание по горизонтали)
box = GtkBox(:h) # :h означает горизонтальное выравнивание

# Создаем несколько кнопок
button1 = GtkButton("Кнопка 1")
button2 = GtkButton("Кнопка 2")
button3 = GtkButton("Кнопка 3")

# Добавляем кнопки в коробку
push!(box, button1, button2, button3)

# Добавляем коробку в окно
push!(win, box)

# Делаем кнопку расширяемой
set_gtk_property!(box, :expand, button3, true)

# Добавляем отступы между виджетами в GtkBox
set_gtk_property!(box, :spacing, 10)

# Показываем окно
showall(win)
```

```
Out[18]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1,000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="GtkBox пример", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=400, default-height=200, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```

9.2.2 GtkButtonBox

Если вы хотите, чтобы кнопки в GtkBox имели равный размер и правильное выравнивание, лучше использовать **GtkButtonBox**. Этот виджет предназначен специально для создания горизонтальных или вертикальных групп кнопок.

```
In [19]: using Gtk

win = GtkWindow("Пример GtkButtonBox", 400, 200)

# Создаем GtkButtonBox с горизонтальным расположением
hbox = GtkButtonBox(:h) # :h - это горизонтальное расположение кнопок

# Создаем кнопки
button1 = GtkButton("Кнопка 1")
button2 = GtkButton("Кнопка 2")
button3 = GtkButton("Кнопка 3")

# Добавляем кнопки в GtkButtonBox
push!(win, hbox)
push!(hbox, button1)
push!(hbox, button2)
push!(hbox, button3)

showall(win)
```

```
Out[19]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1.000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexexpand=FALSE, vexpand=FALSE, hexexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Пример GtkButtonBox", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=400, default-height=200, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```

9.2.3 GtkGrid

GtkGrid – это макет, который позволяет размещать другие виджеты в виде сетки с строками и столбцами.

В GtkGrid можно задавать как обычное размещение виджетов, так и задавать их размеры и отступы между ними. Этот макет используется для построения интерфейсов с более сложным расположением элементов, чем, например, вертикальные или горизонтальные контейнеры.

Для добавления виджетов в сетку мы используем индексы строк и столбцов.

```
In [20]: using Gtk

win = GtkWindow("Пример GtkGrid")
# Создаем сетку
g = GtkGrid()

# Создаем несколько кнопок
button1 = GtkButton("Кнопка 1")
button2 = GtkButton("Кнопка 2")
button3 = GtkButton("Кнопка 3")

# Размещение кнопок в сетке
g[1, 1] = button1 # Кнопка 1 в первом столбце и в первой строке
g[2, 1] = button2 # Кнопка 2 во втором столбце и в первой строке
g[1, 2] = button3 # Кнопка 3 в первом столбце и во второй строке

# Отображаем окно с кнопками
```

```
push!(win, g)
showall(win)
```

```
Out[20]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible
=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1.000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexexpand=FALSE, vexpand=FALSE, hexexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Пример GtkGrid", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=-1, default-height=-1, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```

Свойства GtkGrid:

- **row_spacing** – промежуток между строками;
- **column_spacing** – промежуток между столбцами;
- **column_homogeneous** – если установлено в true, все столбцы будут одинаковой ширины;
- **row_homogeneous** – если установлено в true, все строки будут одинаковой высоты.

```
In [21]: using Gtk
```

```
win = GtkWindow("Пример GtkGrid")
# Создаем сетку
g = GtkGrid()

# Создаем несколько кнопок
button1 = GtkButton("Кнопка 1")
button2 = GtkButton("Кнопка 2")
button3 = GtkButton("Кнопка 3")
button4 = GtkButton("Кнопка 4")

# Размещение виджетов в сетке
g[1, 1] = button1 # button1 в ячейке (1, 1)
g[2, 1] = button2 # button2 в ячейке (2, 1)
g[1, 2] = button3 # button3 в ячейке (1, 2)
g[2, 2] = button4 # button4 в ячейке (2, 2)

# Настройка свойств GtkGrid
set_gtk_property!(g, :row_spacing, 10) # Промежуток между строками
set_gtk_property!(g, :column_spacing, 15) # Промежуток между столбцами
set_gtk_property!(g, :column_homogeneous, true) # Все столбцы одинаковой ширины
```

```

set_gtk_property!(g, :row_homogeneous, true)    # Все строки одинаковой выс
# Добавляем сетку в окно
push!(win, g)

showall(win)

```

```

Out[21]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible
=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALS
E, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALS
E, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-
all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, windo
w, opacity=1,000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALI
GN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-to
p=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=F
ALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resi
ze-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Пример GtkGrid", role=NUL
L, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-w
idth=-1, default-height=-1, destroy-with-parent=FALSE, hide-titlebar-when-m
aximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HIN
T_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALS
E, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gr
avity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip,
resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALS
E, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=F
ALSE)

```

Можно задать несколько ячеек для одного виджета.

```

In [22]: using Gtk

win = GtkWindow("Пример GtkGrid")
# Создаем сетку
g = GtkGrid()

# Создаем несколько кнопок
button1 = GtkButton("Кнопка 1")
button2 = GtkButton("Кнопка 2")
button3 = GtkButton("Кнопка 3")

# Размещение кнопок в сетке
g[1:2, 1] = button1 # button1 займет 2 столбца в 1 строке
g[3, 1:3] = button2 # button2 займет 3 строки в 3 столбце
g[1:3, 4] = button3 # button3 займет 3 столбца в 4 строке

# Настроим расстояние между столбцами и строками
set_gtk_property!(g, :column_spacing, 10)
set_gtk_property!(g, :row_spacing, 10)

# Отображаем окно с кнопками
push!(win, g)
showall(win)

```

```
Out[22]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1.000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Пример GtkGrid", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=-1, default-height=-1, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```

9.3 Текстовые поля

В GTK на языке Julia два наиболее распространённых виджета для работы с текстом – это **GtkLabel** и **GtkEntry**. Оба виджета используются для работы с текстом, но их функциональность различна.

9.3.1 GtkLabel

GtkLabel – метка (текст, который не редактируется)

GtkLabel представляет собой виджет для отображения текста, который не может быть отредактирован пользователем. Это полезно, например, для вывода статического текста, инструкций, заголовков и других элементов, которые должны быть видны, но не изменяемы.

```
In [23]: using Gtk

# Создаём окно
win = GtkWindow("Пример GtkLabel", 400, 200)

# Создаём метку с текстом
label = GtkLabel("Это метка с текстом!")

push!(win, label)
showall(win)
```

```
Out[23]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible
=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE,
is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE,
receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-
all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, windo
w, opacity=1,000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALI
GN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-to
p=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=F
ALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resi
ze-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Пример GtkLabel", role=NUL
L, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-w
idth=400, default-height=200, destroy-with-parent=FALSE, hide-titlebar-when
-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_H
INT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FA
LSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE,
gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-gri
p, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FA
LSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized
=FALSE)
```

Текст надписи можно изменить с помощью **GAccessor.text**.

```
In [24]: using Gtk
```

```
win = GtkWindow("Пример GtkLabel", 400, 200)

label = GtkLabel("Это метка с текстом!")
GAccessor.text(label, "Мой другой текст")

push!(win, label)
showall(win)
```

```
Out[24]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible
=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE,
is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE,
receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-
all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, windo
w, opacity=1,000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALI
GN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-to
p=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=F
ALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resi
ze-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Пример GtkLabel", role=NUL
L, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-w
idth=400, default-height=200, destroy-with-parent=FALSE, hide-titlebar-when
-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_H
INT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FA
LSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE,
gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-gri
p, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FA
LSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized
=FALSE)
```

Метод **GAccessor.markup** используется для того, чтобы задать разметку (markup) для текста в виджете GtkLabel. Это позволяет не просто

отображать текст, но и форматировать его, например, сделать части текста жирными, курсивными, добавить гиперссылки или изменить цвет.

Давайте рассмотрим примеры, где будет использована разметка для стилизации текста.

Пример 1. Изменение размера текста (size).

In [25]: **using** Gtk

```
# Создаем окно
win = GtkWindow("Изменение размера шрифта", 400, 300)

label = GtkLabel("")
GAccessor.markup(label, ""
<span size="small">Маленький размер шрифта</span>\n
<span size="medium">Средний размер шрифта</span>\n
<span size="large">Большой размер шрифта</span>\n
<span size="x-large">Очень большой размер шрифта</span>\n
<span size="xx-large">Очень-очень большой размер шрифта</span>
""")

# Добавляем метку в окно
push!(win, label)

# Показываем окно
showall(win)
```

Out[25]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1.000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Изменение размера шрифта", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=400, default-height=300, destroy-with-parent=FALSE, hide-title-bar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)

Пример 2. Установка цвета текста (foreground).

In [26]: **using** Gtk

```
# Создаем окно
win = GtkWindow("Цвет текста", 400, 300)
```

```

label = GtkLabel("")
GAccessor.markup(label, ""
<span foreground="red">Красный цвет текста</span>\n
<span foreground="green">Зеленый цвет текста</span>\n
<span foreground="blue">Синий цвет текста</span>\n
<span foreground="purple">Пурпурный цвет текста</span>\n
<span foreground="#FF5733">Пользовательский цвет (#FF5733)</span>
""")

push!(win, label)
showall(win)

```

```

Out[26]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible
=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE,
is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE,
receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-
all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, windo
w, opacity=1,000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALI
GN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-to
p=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=F
ALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resi
ze-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Цвет текста", role=NULL, r
esizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width
=400, default-height=300, destroy-with-parent=FALSE, hide-titlebar-when-max
imized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_
NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE,
accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravi
ty=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, res
ize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, s
tartup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)

```

Пример 3. Установка фона текста (background).

```

In [27]: using Gtk

# Создаем окно
win = GtkWindow("Цвет фона текста", 400, 300)

label = GtkLabel("")
GAccessor.markup(label, ""
<span background="yellow" foreground="black">Желтый фон и черный текст </spa
<span background="lightgray" foreground="black">Светло-серый фон и черный те
<span background="cyan" foreground="black">Голубой фон и черный текст </spa
<span background="black" foreground="white">Черный фон и белый текст</span>
""")

# Добавляем метку в окно
push!(win, label)

# Показываем окно
showall(win)

```

```
Out[27]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1.000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexexpand=FALSE, vexpand=FALSE, hexexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Цвет фона текста", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=400, default-height=300, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```

Пример 4. Использование конкретного шрифта (font).

```
In [28]: using Gtk

# Создаем окно
win = GtkWindow("Шрифт текста", 400, 300)

label = GtkLabel("")
GAccessor.markup(label, ""
<span font="Arial 14">Шрифт Arial, размер 14</span>\n
<span font="Courier New 18">Шрифт Courier New, размер 18</span>\n
<span font="Times New Roman 20">Шрифт Times New Roman, размер 20</span>\n
<span font="Helvetica 16">Шрифт Helvetica, размер 16</span>
"")

# Добавляем метку в окно
push!(win, label)

# Показываем окно
showall(win)
```

```
Out[28]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1.000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexexpand=FALSE, vexpand=FALSE, hexexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Шрифт текста", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=400, default-height=300, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```

Пример 5. Изменение жирности шрифта (weight).

```
In [29]: using Gtk

# Создаем окно
win = GtkWindow("Жирность текста", 400, 300)

label = GtkLabel("")
GAccessor.markup(label, ""
<span weight="normal">Обычный шрифт</span>\n
<span weight="bold"> Жирный шрифт</span>\n
<span weight="light">Легкий шрифт</span>\n
""")

# Добавляем метку в окно
push!(win, label)

# Показываем окно
showall(win)
```

```
Out[29]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1.000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexexpand=FALSE, vexpand=FALSE, hexexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Жирность текста", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=400, default-height=300, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```

Пример 6. Изменение стиля шрифта (style).

```
In [30]: using Gtk

# Создаем окно
win = GtkWindow("Стиль текста", 400, 300)

label = GtkLabel("")
GAccessor.markup(label, ""
<span style="normal">Нормальный стиль</span>\n
<span style="italic">Курсивный стиль</span>\n
<span style="oblique">Наклонный стиль</span>\n
""")

# Добавляем метку в окно
push!(win, label)

# Показываем окно
showall(win)
```

```
Out[30]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1.000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Стиль текста", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=400, default-height=300, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```

Пример 7. Подчеркивание текста (underline).

```
In [31]: using Gtk

# Создаем окно
win = GtkWindow("Подчеркивание текста", 400, 300)

label = GtkLabel("")
GAccessor.markup(label, ""
<span underline="none">Без подчеркивания</span>\n
<span underline="single">Одиночное подчеркивание</span>\n
<span underline="double">Двойное подчеркивание</span>\n
<span underline="error">Подчеркивание ошибочного текста</span>
""")

# Добавляем метку в окно
push!(win, label)

# Показываем окно
showall(win)
```

```
Out[31]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1,000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Подчеркивание текста", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=400, default-height=300, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```

Пример 8. Перечеркивание текста (strikethrough).

```
In [33]: using Gtk

# Создаем окно
win = GtkWindow("Перечеркивание текста", 400, 300)

label = GtkLabel("")
GAccessor.markup(label, ""
<span strikethrough="false">Без перечеркивания</span>\n
<span strikethrough="true">Перечеркнутый текст</span>
""")

# Добавляем метку в окно
push!(win, label)

# Показываем окно
showall(win)
```

```
Out[33]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1,000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Перечеркивание текста", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=400, default-height=300, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```

9.3.2 GtkEntry

GtkEntry – это виджет для ввода текста, который представляет собой однострочное текстовое поле.

Основные свойства GtkEntry:

- **:text** – текущее содержимое текстового поля (строка);
- **:visibility** – отображение текста (используется, например, для скрытия текста в поле ввода пароля);
- **:editable** – разрешает или запрещает редактирование текста;
- **:placeholder_text** – текст-подсказка, который отображается в поле, если оно пустое (например, для указания пользователю, что нужно ввести).

Пример 1. Создание простого поля для ввода текста.

```
In [34]: using Gtk

# Создаем окно
win = GtkWindow("GtkEntry")

# Создаем поле для ввода текста
entry = GtkEntry()

# Устанавливаем начальный текст
set_gtk_property!(entry, :text, "Введите текст сюда")

# Добавляем поле в окно
push!(win, entry)
showall(win)
```

```
Out[34]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1,000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="GtkEntry", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=-1, default-height=-1, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```

Пример 2. Получение текста из поля ввода.

```
In [36]: using Gtk
```

```
# Создаем окно
win = GtkWindow("GtkEntry",400,300)

# Создаем поле для ввода текста
entry = GtkEntry()

# Устанавливаем текст по умолчанию
set_gtk_property!(entry, :placeholder_text, "Введите ваш email")

# Функция для получения введенного текста
function get_input_text(widget)
    str = get_gtk_property(entry, :text, String)
    println("Введенный текст: ", str)
end

# Добавляем обработчик события для нажатия клавиши Enter
signal_connect(get_input_text, entry, "activate")

# Добавляем поле в окно
push!(win, entry)
showall(win)
```

```
Out[36]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible
=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE,
is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE,
receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE,
has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1,000000,
double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right,
margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexpand=FALSE,
vexpand=FALSE, hexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1,
border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="GtkEntry",
role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE,
default-width=400, default-height=300, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE,
icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE,
skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE,
decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for,
attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE,
has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE,
is-maximized=FALSE)
Введенный текст: julia
```

Пример 3. Работа с паролем (скрытие текста).

```
In [37]: using Gtk

# Создаем окно
win = GtkWindow("Password", 500, 400)

# Создаем поле для ввода пароля
entry = GtkEntry()

# Устанавливаем текст-подсказку
set_gtk_property!(entry, :placeholder_text, "Введите ваш пароль")

# Скрываем введенный текст (для поля пароля)
set_gtk_property!(entry, :visibility, false)

# Добавляем поле в окно
push!(win, entry)
showall(win)
```

```
Out[37]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1,000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Password", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=500, default-height=400, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```

Пример 4. Отключение редактирования.

```
In [38]: using Gtk

# Создаем окно
win = GtkWindow("Пример записи, не подлежащей редактированию", 500, 400)

# Создаем поле для ввода текста
entry = GtkEntry()

# Устанавливаем начальный текст
set_gtk_property!(entry, :text, "Этот текст нельзя редактировать")

# Отключаем редактирование
set_gtk_property!(entry, :editable, false)

# Добавляем поле в окно
push!(win, entry)
showall(win)
```

```
Out[38]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1,000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Пример записи, не подлежащей редактированию", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=500, default-height=400, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```

9.4 Виджеты списка и дерева

GtkTreeView – это виджет для отображения табличных или иерархических данных. Несмотря на название, **GtkTreeView** используется не только для отображения деревьев, но и для работы с обычными списками. Главной особенностью этих виджетов является то, что **GtkTreeView** не хранит данные напрямую. Вместо этого данные хранятся в контейнерах, таких как **GtkListStore** для списков и **GtkTreeStore** для деревьев.

Списки (или таблицы) представляют собой упорядоченные коллекции данных, где каждый элемент можно воспринимать как строку. Например, можно создать таблицу, в которой будут храниться данные о людях: имя, возраст и пол. Для таких данных удобно использовать **GtkListStore**, который представляет собой таблицу, где каждая строка может содержать несколько значений разных типов. Если же необходимо отобразить данные в виде дерева, где элементы могут иметь дочерние элементы, то лучше использовать **GtkTreeStore**. Дерево позволяет организовать элементы в иерархическую структуру, где один элемент может быть родителем других, создавая, например, структуру каталогов в файловой системе. В обоих случаях для отображения данных используется виджет **GtkTreeView**, который привязывается к контейнеру данных и отображает их на экране. Отличие между списком и деревом заключается в том, что в списке все элементы независимы друг от друга, а в дереве каждый элемент может иметь потомков, создавая иерархию.

9.4.1 GtkListStore

Создание GtkListStore: `store = GtkListStore(T1, T2, ..., Tn),`

где **T1, T2, ..., Tn** – типы данных для каждого столбца. Например, `String`, `Int`, `Bool`, и т.д.

Добавление строки: `push!(store, (value1, value2, ..., value_n))`

- **store** – объект `GtkListStore`;
- **value1, value2, ..., value_n** – значения для добавления в строку.

Вставка данных: `insert!(store, row_index, (value1, value2, ..., value_n))`

- **store** – это объект `GtkListStore`;
- **row_index** – индекс строки, в которую нужно вставить данные;
- **(value1, value2, ..., value_n)** – кортеж значений, которые будут вставлены в соответствующие столбцы.

Получение данных с использованием индексации:

- `ls[row_index, column_index]` – возвращает строку по индексу;
- `ls[row_index, column_index] = value` – устанавливает значение в строку и столбец по индексу.

Получение количества строк: `length(store)`

GtkTreeView – это виджет, который отображает данные из модели в виде таблицы или дерева. Важный момент: для отображения данных в `GtkTreeView` нам нужно передать модель данных через интерфейс `GtkTreeModel`. `GtkTreeView` получает данные из модели и использует их для отображения на экране.

`GtkTreeView(GtkTreeModel(store))`

Рендереры – это компоненты, которые отвечают за визуализацию данных, предоставленных моделью, в виде видимых элементов на экране. Они могут отображать текст, изображения, прогресс-бары, чекбоксы и другие элементы интерфейса.

GtkCellRendererText – это рендерер, который используется для отображения текста в ячейках таблицы или дерева. Когда вы хотите отображать строковые данные (например, имя или возраст), вам нужно использовать этот рендерер.

GtkCellRendererToggle – это рендерер, который используется для отображения булевых значений в виде чекбоксов. Когда значение в модели данных представляет собой `true` или `false`, этот рендерер позволяет пользователю взаимодействовать с этим значением через чекбокс.

GtkCellRendererProgress – это рендерер, который используется для отображения прогресса выполнения в ячейках таблицы или дерева. Он принимает числовое значение и отображает его как прогресс (например, от 0% до 100%).

GtkTreeViewColumn – это объект, который определяет, как отображать данные в каждой колонке, ассоциируя их с рендерерами, которые отвечают за визуальное представление данных (например, текст, чекбоксы, прогресс-бары и т.д.).

Пример 1. Список людей с именем, возрастом и полом.

In [39]: **using** Gtk

```
# Создаем контейнер для данных: 3 колонки (имя, возраст, пол)
ls = GtkListStore(String, Int, Bool)

# Заполняем список данными
push!(ls, ("Саша", 20, false)) # Мужчина
push!(ls, ("Рома", 30, false)) # Мужчина
push!(ls, ("Маша", 25, true))  # Женщина

# Создаем виджет TreeView для отображения данных из контейнера
tv = GtkTreeView(GtkTreeModel(ls))

# Создаем рендереры для отображения данных в колонках. Рендереры – это компоненты
rTxt = GtkCellRendererText() # Рендерер для текста
rTog = GtkCellRendererToggle() # Рендерер для чекбокса

# Создаем колонки для отображения данных
c1 = GtkTreeViewColumn("Имя", rTxt, Dict{String, Int}([("text", 0)])) # Колонка для имени
c2 = GtkTreeViewColumn("Возраст", rTxt, Dict{String, Int}([("text", 1)])) # Колонка для возраста
c3 = GtkTreeViewColumn("Пол", rTog, Dict{String, Int}([("active", 2)])) # Колонка для пола
# Добавляем колонки в TreeView

push!(tv, c1, c2, c3)

# Создаем окно для отображения данных
win = GtkWindow(tv, "List View")

# Показываем окно
showall(win)
```

```
Out[39]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1,000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="List View", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=-1, default-height=-1, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```

Пример 2. Список людей с именем, возрастом и полом, с возможностью изменять ширину колонок.

```
In [45]: using Gtk
```

```
# Создаем контейнер для данных: 3 колонки (имя, возраст, пол)
ls = GtkListStore{String, Int, Bool}

# Заполняем список данными
push!(ls, ("Саша", 20, false)) # Мужчина
push!(ls, ("Рома", 30, false)) # Мужчина
push!(ls, ("Маша", 25, true))  # Женщина

# Создаем виджет TreeView для отображения данных из контейнера
tv = GtkTreeView(GtkTreeModel{ls})

# Создаем рендереры для отображения данных в колонках. Рендереры – это композиты
rTxt = GtkCellRendererText() # Рендерер для текста
rTog = GtkCellRendererToggle() # Рендерер для чекбокса

# Создаем колонки для отображения данных
c1 = GtkTreeViewColumn("Имя", rTxt, Dict{String, Int}([("text", 0)])) # Колонка для имени
c2 = GtkTreeViewColumn("Возраст", rTxt, Dict{String, Int}([("text", 1)])) # Колонка для возраста
c3 = GtkTreeViewColumn("Пол", rTog, Dict{String, Int}([("active", 2)])) # Колонка для пола
# Добавляем колонки в TreeView

push!(tv, c1, c2, c3)

for c in [c1, c2, c3]
    GAccessor.resizable(c, true)
#функция задает, может ли колонка быть изменена по ширине пользователем.
#Параметр true говорит о том, что колонка будет изменяемой по ширине.
#Если бы был передан параметр false, колонка не могла бы изменять свою ширину
end
```

```
# Создаем окно для отображения данных
win = GtkWindow(tv, "List View")

# Показываем окно
showall(win)
```

```
Out[45]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=
TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1.000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexexpand=FALSE, vexpand=FALSE, hexexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="List View", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=-1, default-height=-1, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```

Пример 3. Отображение прогресса задач.

```
In [42]: using Gtk

# Создаём модель данных для имени и прогресса
ls = GtkListStore{String, Int}

# Добавляем данные в модель (имя и прогресс)
push!(ls, ("Задача 1", 50)) # Прогресс 50%
push!(ls, ("Задача 2", 80)) # Прогресс 80%
push!(ls, ("Задача 3", 30)) # Прогресс 30%

# Создаём виджет GtkTreeView с моделью данных
tv = GtkTreeView(GtkTreeModel{ls})

# Создаём рендерер для отображения прогресса
rProg = GtkCellRendererProgress()

# Создаём колонку для отображения прогресса
c1 = GtkTreeViewColumn("Задача", GtkCellRendererText(), Dict{("text", 0)})
c2 = GtkTreeViewColumn("Прогресс", rProg, Dict{("value", 1)}) # "value" y

# Добавляем колонки в GtkTreeView
push!(tv, c1, c2)

# Создаём окно и отображаем виджет
win = GtkWindow(tv, "Таблица с прогрессом")
showall(win)
```

```
Out[42]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible
=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE,
is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE,
receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE,
has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1.000000,
double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left,
margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0,
margin=0, hexexpand=FALSE, vexpand=FALSE, hexexpand-set=FALSE, vexpand-set=FALSE,
expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL,
title="Таблица с прогнозом", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE,
default-width=-1, default-height=-1, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE,
icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE,
skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE,
deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip,
resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE,
startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```

Когда мы работаем с таблицами или списками данных в GTK, часто возникает необходимость взаимодействовать с выбранным элементом. Для этого используется объект **GtkTreeSelection**, который позволяет отслеживать и управлять выбором элементов в списке. Чтобы получить этот объект, необходимо использовать функцию:

```
selection = GAccessor.selection(tv)
```

Здесь **tv** – это объект **GtkTreeView**, в котором отображаются данные. С помощью **GtkTreeSelection** мы можем задать режим выбора: один элемент или несколько. В данном примере мы будем использовать выбор только одного элемента (одиночный выбор). Для включения мульти-выбора можно вызвать:

```
selection = GAccessor.mode(selection,
Gtk.GConstants.GtkSelectionMode.MULTIPLE)
```

Для текущего примера мы будем использовать одиночный выбор, и нам нужно получить индекс выбранного элемента. Это можно сделать следующим образом:

```
selected_item = selected(selection)
println("Выбранный элемент: ", ls[selected_item, 1])
```

Здесь **selected(selection)** возвращает индекс выбранного элемента, и мы можем получить данные из соответствующей строки с помощью индексации.

В случае, если пользователь выбрал элемент и вы хотите выполнить какое-то действие, например, вывести информацию о выбранной строке, можно использовать сигнал **"changed"**, который срабатывает каждый раз при изменении выбора:

```

signal_connect(selection, "changed") do widget
  if hasselection(selection)
    currentIt = selected(selection)
    println("Имя: ", ls[currentIt, 1], " Возраст: ", ls[currentIt,
2])
  end
end
end

```

Этот код подключает обработчик события, который будет вызываться каждый раз, когда пользователь выбирает новый элемент. В нем проверяется, был ли выбран элемент (с помощью **hasselection(selection)**), и если да – выводится информация о выбранной строке.

In [46]: **using** Gtk

```

# Создаем контейнер для данных: 3 колонки (имя, возраст, пол)
ls = GtkListStore(String, Int, Bool)

# Заполняем список данными
push!(ls, ("Саша", 20, false)) # Мужчина
push!(ls, ("Рома", 30, false)) # Мужчина
push!(ls, ("Маша", 25, true))  # Женщина

# Создаем виджет TreeView для отображения данных из контейнера
tv = GtkTreeView(GtkTreeModel(ls))

# Создаем рендереры для отображения данных в колонках.
rTxt = GtkCellRendererText() # Рендерер для текста
rTog = GtkCellRendererToggle() # Рендерер для чекбокса

# Создаем колонки для отображения данных
c1 = GtkTreeViewColumn("Имя", rTxt, Dict([("text", 0)])) # Колонка для имен
c2 = GtkTreeViewColumn("Возраст", rTxt, Dict([("text", 1)])) # Колонка для
c3 = GtkTreeViewColumn("Пол", rTog, Dict([("active", 2)])) # Колонка для по
# Добавляем колонки в TreeView

push!(tv, c1, c2, c3)

# Создаем окно для отображения данных
win = GtkWindow(tv, "List View")

selection = GAccessor.selection(tv)
signal_connect(selection, "changed") do widget
  if hasselection(selection)
    currentIt = selected(selection)
    println("Выбранный элемент: ", ls[currentIt, 1], " Возраст: ", ls[curren
  end
end

# Показываем окно
showall(win)

```

```
Out[46]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1,000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="List View", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=-1, default-height=-1, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
Выбранный элемент: Рома Возраст: 30
Выбранный элемент: Саша Возраст: 20
```

9.4.2 GtkTreeStore

Пример 1. Создание дерева с использованием GtkTreeStore.

```
In [48]: using Gtk

# Создание модели дерева, в которой будут храниться строки
ts = GtkTreeStore{String}

# Добавляем элементы в дерево
iter1 = push!(ts, ("1",))
iter2 = push!(ts, ("2",), iter1)
iter3 = push!(ts, ("3",), iter2)

# Создаем виджет для отображения дерева
tv = GtkTreeView(GtkTreeModel{ts})

# Создаем рендерер текста для отображения текста в столбце
r1 = GtkCellRendererText()

# Создаем колонку для отображения данных
c1 = GtkTreeViewColumn("A", r1, Dict{String, Any}([("text", 0)]))

# Добавляем колонку в TreeView
push!(tv, c1)

# Создаем окно с TreeView
win = GtkWindow(tv, "Tree View")

# Показываем окно
showall(win)

# Изменение текста в первом элементе дерева
```

```
iter = Gtk.iter_from_index(ts, [1])
ts[iter, 1] = "один"
```

Out[48]: "один"

9.5 События клавиш

Чтобы обрабатывать события нажатия клавиш, необходимо использовать событие **key-press-event** для активного окна. Это событие срабатывает каждый раз, когда пользователь нажимает клавишу на клавиатуре в пределах этого окна. Ниже приведен пример кода, который демонстрирует, как это сделать.

```
In [50]: using Gtk
win = GtkWindow("Пример нажатия клавиш")

# Подключаем обработчик события нажатия клавиши
signal_connect(win, "key-press-event") do widget, event
  # Извлекаем значение нажатой клавиши
  k = event.keyval

  # Выводим на экран код клавиши и сам символ
  println("Вы нажали клавишу с кодом ", k, ", что соответствует символу '",
end

showall(win)
```

```
Out[50]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1,000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexexpand=FALSE, vexpand=FALSE, hexexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Пример нажатия клавиш", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=-1, default-height=-1, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
Вы нажали клавишу с кодом 101, что соответствует символу 'e'.
Вы нажали клавишу с кодом 99, что соответствует символу 'c'.
Вы нажали клавишу с кодом 103, что соответствует символу 'g'.
```

9.6 Рисование в GTK

С помощью библиотеки Graphics.jl в сочетании с GTK.jl можно рисовать на **канвасе** (Canvas). Канвас в контексте графических интерфейсов – это специальный элемент (виджет), на котором можно рисовать различные графические объекты, такие как линии, прямоугольники, окружности, текст, изображения и так далее.

@GtkCanvas() – это макрос, создающий канвас для рисования.

```
canvas = @GtkCanvas()
```

@guarded draw() – этот макрос используется для создания обработчика рисования, который будет вызываться каждый раз, когда нужно перерисовать канвас (например, при изменении размеров окна или обновлении содержимого).

```
@guarded draw(widget) do widget
    # Код рисования
end
```

Где widget – это объект, на котором происходит рисование (в данном случае канвас).

getgc() – функция, которая возвращает контекст рисования для канваса. Контекст рисования – это объект, который управляет такими параметрами, как цвет, шрифт и стиль линии.

```
ctx = getgc(canvas)
```

Где canvas – объект канваса, с которого нужно получить контекст, а getgc(canvas) возвращает объект контекста рисования ctx, который используется для выполнения операций рисования, таких как рисование линий, фигур и настройка цвета.

height() и **width()** – эти функции возвращают размеры канваса, которые полезны для адаптивного рисования в зависимости от размеров окна.

```
h = height(canvas)
```

```
w = width(canvas)
```

Где height(canvas) – возвращает высоту канваса canvas, а width(canvas) – возвращает ширину канваса canvas.

rectangle() – рисует прямоугольник в контексте рисования. Этот вызов не заполняет прямоугольник, а только рисует его контур.

```
rectangle(ctx, x, y, w, h)
```

Где x, y – координаты верхнего левого угла прямоугольника, а w, h – ширина и высота прямоугольника.

set_source_rgb() – задает цвет рисования с использованием модели RGB (красный, зеленый, синий).

```
set_source_rgb(ctx, r, g, b)
```

Где r, g, b – компоненты красного, зеленого и синего цвета в диапазоне от 0 до 1.

fill() – заполняет текущую форму (например, прямоугольник) текущим цветом.

```
fill(ctx)
```

Эта функция применяет текущий цвет и заполняет все фигуры, нарисованные до этого (например, прямоугольники, круги).

In [52]: **using** Gtk, Graphics

```
# Создаем канвас для рисования
c = @GtkCanvas()

# Создаем окно с канвасом
win = GtkWindow(c, "Canvas")

# Гарантируем, что рисунок будет перерисован
@guarded draw(c) do widget
    # Получаем контекст рисования (gc) для канваса
    ctx = getgc(c)

    # Получаем размеры канваса
    h = height(c)
    w = width(c)

    # Рисуем первый красный прямоугольник (верхнюю половину окна)
    rectangle(ctx, 0, 0, w, h / 2)
    set_source_rgb(ctx, 1, 0, 0) # Красный цвет
    fill(ctx) # Заполняем прямоугольник красным цветом

    # Рисуем второй синий прямоугольник (нижнюю четверть окна)
    rectangle(ctx, 0, 3 * h / 4, w, h / 4)
    set_source_rgb(ctx, 0, 0, 1) # Синий цвет
    fill(ctx) # Заполняем прямоугольник синим цветом
end

# Отображаем окно с канвасом
show(c)
```

Out[52]: GtkCanvas(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=GDK_POINTER_MOTION_MASK | GDK_POINTER_MOTION_HINT_MASK | GDK_BUTTON_PRESS_MASK | GDK_BUTTON_RELEASE_MASK | GDK_SCROLL_MASK, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1,000000, double-buffered, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1)

Рисование с обработкой событий мыши

Теперь давайте добавим возможность рисовать на канвасе с помощью мыши. Каждый раз, когда пользователь нажимает кнопку мыши (например, левую кнопку), на канвасе будет рисоваться зеленый круг в том месте, где произошел клик.

```
In [3]: using Gtk, Graphics

# Создаем канвас
c = @GtkCanvas()

# Создаем окно с канвасом
win = GtkWindow(c, "Canvas")

# Подключаем обработчик события нажатия левой кнопки мыши.(button2press – со
c.mouse.button1press = @guarded (widget, event) -> begin
    # Получаем контекст рисования
    ctx = getgc(widget)

    # Устанавливаем зеленый цвет для рисования
    set_source_rgb(ctx, 0, 1, 0)

    # Рисуем круг в точке, где был клик мышью
    arc(ctx, event.x, event.y, 5, 0, 2pi) # arc(ctx, xc, yc, radius, angle1,
    #ctx: графический контекст полученный с помощью getgc(widget). Это объект
    #xc, yc: координаты центра дуги или круга.
    #radius: радиус дуги или круга.
    #angle1, angle2: углы, определяющие начало и конец дуги.
    #Углы измеряются в радианах, и если angle1 и angle2 охватывают полный кр
    #Если же углы не охватывают весь круг, будет нарисована дуга.
    stroke(ctx) # Обводим круг

    # Обновляем канвас, чтобы отобразить изменения
    reveal(widget)
end

# Показываем окно
show(c)
```

```
Out[3]: GtkCanvas(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=GDK_POINTER_MOTION_MASK | GDK_POINTER_MOTION_HINT_MASK | GDK_BUTTON_PRESS_MASK | GDK_BUTTON_RELEASE_MASK | GDK_SCROLL_MASK, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1.000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1)
```

Пример с добавлением прямоугольника и линии.

In [4]: **using** Gtk

```
# Создаем канвас
c = @GtkCanvas()

# Создаем окно с канвасом
win = GtkWindow(c, "Canvas")

# Подключаем обработчик события нажатия левой кнопки мыши
c.mouse.button1press = @guarded (widget, event) -> begin
    # Получаем контекст рисования
    ctx = getgc(widget)

    # Рисуем круг с радиусом 20
    set_source_rgb(ctx, 0, 1, 0) # Зеленый цвет
    arc(ctx, event.x, event.y, 20, 0, 2pi)
    stroke(ctx) # Обводим круг

    # Рисуем прямоугольник
    set_source_rgb(ctx, 1, 0, 0) # Красный цвет
    rectangle(ctx, event.x + 40, event.y + 40, 60, 30) # Команда rectangle(c
    #ctx: графический контекст, полученный через getgc(widget). Это объект,
    #x, y: координаты верхнего левого угла прямоугольника. Эти значения опре
    #width, height: размеры прямоугольника: ширина и высота соответственно.
    stroke(ctx) # Обводим прямоугольник

    # Рисуем линию
    set_source_rgb(ctx, 0, 0, 1) # Синий цвет
    move_to(ctx, event.x, event.y) # move_to используется для перемещения кур
    line_to(ctx, event.x + 100, event.y + 100) #line_to(ctx, x, y) используе
    #ctx: графический контекст.
    #x, y: координаты конечной точки линии.

    # Обновляем канвас, чтобы отобразить изменения
    reveal(widget)
end

# Показываем окно
show(c)
```

Out[4]: GtkCanvas(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=GDK_POINTER_MOTION_MASK | GDK_POINTER_MOTION_HINT_MASK | GDK_BUTTON_PRESS_MASK | GDK_BUTTON_RELEASE_MASK | GDK_SCROLL_MASK, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1.000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, hexpand=FALSE, vexpand=FALSE, hexpand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1)

9.7 Пример программы

In [5]: **using** Gtk

```
# Функция для обновления текста в поле ввода
function update_display(entry, text)
    GAccessor.text(entry, text)
end

# Функция для обработки нажатий кнопок
function on_button_click(entry, label)
    str = get_gtk_property(entry, :text, String)
    new_text = str * label
    update_display(entry, new_text)
end

# Функция для вычисления результата
function calculate_result(entry)
    str = get_gtk_property(entry, :text, String)
    result = eval(Meta.parse(str))
    str = "$result"
    update_display(entry, str)
end

# Функция для очистки поля ввода
function clear_entry(entry)
    update_display(entry, "")
end

# Основная функция для создания окна калькулятора
function create_calculator_window()
    win = GtkWindow("Калькулятор", 150, 150)

    # Создаем поле для ввода текста (где отображаются числа и операторы)
    entry = GtkEntry()
    set_gtk_property!(entry, :editable, false)
    GAccessor.text(entry, "")
    box = GtkBox(:v)
    push!(box, entry)

    # Создаем сетку для размещения кнопок
    grid = GtkGrid()

    # Создаем кнопки и привязываем обработчики
    button7 = GtkButton("7")
    signal_connect(button7, :clicked) do widget
        on_button_click(entry, "7")
    end

    button8 = GtkButton("8")
    signal_connect(button8, :clicked) do widget
        on_button_click(entry, "8")
    end

    button9 = GtkButton("9")
    signal_connect(button9, :clicked) do widget
        on_button_click(entry, "9")
```

```

end

button_div = GtkButton("/")
signal_connect(button_div, :clicked) do widget
    on_button_click(entry, "/")
end

button4 = GtkButton("4")
signal_connect(button4, :clicked) do widget
    on_button_click(entry, "4")
end

button5 = GtkButton("5")
signal_connect(button5, :clicked) do widget
    on_button_click(entry, "5")
end

button6 = GtkButton("6")
signal_connect(button6, :clicked) do widget
    on_button_click(entry, "6")
end

button_mul = GtkButton("*")
signal_connect(button_mul, :clicked) do widget
    on_button_click(entry, "*")
end

button1 = GtkButton("1")
signal_connect(button1, :clicked) do widget
    on_button_click(entry, "1")
end

button2 = GtkButton("2")
signal_connect(button2, :clicked) do widget
    on_button_click(entry, "2")
end

button3 = GtkButton("3")
signal_connect(button3, :clicked) do widget
    on_button_click(entry, "3")
end

button_sub = GtkButton("-")
signal_connect(button_sub, :clicked) do widget
    on_button_click(entry, "-")
end

button0 = GtkButton("0")
signal_connect(button0, :clicked) do widget
    on_button_click(entry, "0")
end

button_dot = GtkButton(".")
signal_connect(button_dot, :clicked) do widget
    on_button_click(entry, ".")
end

```

```

button_eq = GtkButton("=")
signal_connect(button_eq, :clicked) do widget
    calculate_result(entry)
end

button_add = GtkButton("+")
signal_connect(button_add, :clicked) do widget
    on_button_click(entry, "+")
end

# Создание кнопки для очистки поля ввода
button_clear = GtkButton("C")
signal_connect(button_clear, :clicked) do widget
    clear_entry(entry)
end

# Размещение кнопок в сетке
grid[1, 1] = button7
grid[2, 1] = button8
grid[3, 1] = button9
grid[4, 1] = button_div

grid[1, 2] = button4
grid[2, 2] = button5
grid[3, 2] = button6
grid[4, 2] = button_mul

grid[1, 3] = button1
grid[2, 3] = button2
grid[3, 3] = button3
grid[4, 3] = button_sub

grid[1, 4] = button0
grid[2, 4] = button_dot
grid[3, 4] = button_eq
grid[4, 4] = button_add

# Размещение кнопки очистки
grid[1:4, 5] = button_clear

# Автоматическое подстраивание размеров ячеек
set_gtk_property!(grid, :column_homogeneous, true) # Автоматическое под
set_gtk_property!(grid, :row_homogeneous, true)     # Автоматическое под

# Добавление кнопок в окно
push!(box, grid)
set_gtk_property!(win, :child, box)

# Показать окно
showall(win)
end

# Запуск калькулятора
create_calculator_window()

```

```
Out[5]: GtkWindowLeaf(name="", parent, width-request=-1, height-request=-1, visible=TRUE, sensitive=TRUE, app-paintable=FALSE, can-focus=FALSE, has-focus=FALSE, is-focus=FALSE, focus-on-click=TRUE, can-default=FALSE, has-default=FALSE, receives-default=FALSE, composite-child=FALSE, style, events=0, no-show-all=FALSE, has-tooltip=FALSE, tooltip-markup=NULL, tooltip-text=NULL, window, opacity=1.000000, double-buffered, halign=GTK_ALIGN_FILL, valign=GTK_ALIGN_FILL, margin-left, margin-right, margin-start=0, margin-end=0, margin-top=0, margin-bottom=0, margin=0, expand=FALSE, vexpand=FALSE, expand-set=FALSE, vexpand-set=FALSE, expand=FALSE, scale-factor=1, border-width=0, resize-mode, child, type=GTK_WINDOW_TOPLEVEL, title="Калькулятор", role=NULL, resizable=TRUE, modal=FALSE, window-position=GTK_WIN_POS_NONE, default-width=150, default-height=150, destroy-with-parent=FALSE, hide-titlebar-when-maximized=FALSE, icon, icon-name=NULL, screen, type-hint=GDK_WINDOW_TYPE_HINT_NORMAL, skip-taskbar-hint=FALSE, skip-pager-hint=FALSE, urgency-hint=FALSE, accept-focus=TRUE, focus-on-map=TRUE, decorated=TRUE, deletable=TRUE, gravity=GDK_GRAVITY_NORTH_WEST, transient-for, attached-to, has-resize-grip, resize-grip-visible, application, is-active=FALSE, has-toplevel-focus=FALSE, startup-id, mnemonics-visible=FALSE, focus-visible=FALSE, is-maximized=FALSE)
```